Next.js

Complete Interview Preparation Guide

20 Questions | App Router | Server Components | Performance

Fundamentals	**Q1–Q6**	Rendering, Architecture, Hydration
Advanced Concepts	**Q7–Q12**	Streaming, Data Fetching, Auth, Image Opt.
Expert Topics	**Q13–Q20**	Server Actions, Caching, RBAC, Performance

Q1 What is Next.js and why would you choose it over React?

What is Next.js?

Next.js is a **React framework** built by Vercel that extends React with powerful features for production applications. React is a UI library — it only handles the view layer. Next.js adds the full application infrastructure on top.

Feature	React (alone)	Next.js
Routing	Manual (React Router)	Built-in file-based routing
Rendering	CSR only	SSR, SSG, ISR, CSR — all supported
SEO	Poor (empty HTML)	Excellent (pre-rendered HTML)
Data Fetching	useEffect / libraries	Native async/await in components
API Routes	Separate backend needed	Built-in API routes / Server Actions
Image Optimization	Manual	next/image with auto-optimization
Code Splitting	Manual	Automatic per page
TypeScript	Manual setup	Built-in support

When to choose Next.js:

- You need **SEO** (blogs, e-commerce, marketing sites)
- You want **server-side logic** without a separate backend
- You need **multiple rendering strategies** in one app
- You want **faster initial page load** for users
- You're building a **large production application**

■ **Tip:** Next.js is React — every React skill transfers. It just adds a powerful layer on top.

Q2 Explain the difference between SSR, SSG, ISR, and CSR.

Strategy	When Generated	Best Use Cases	Trade-offs
SSR Server-Side Rendering	Per request (runtime)	Dynamic data, user-specific pages, dashboards	Slower TTFB, server load
SSG Static Site Gen.	At build time (once)	Blogs, docs, marketing pages, content sites	Stale data, slow builds at scale
ISR Incremental Static Regen.	Build + interval (background)	E-commerce, news, pages with periodic updates	Possible brief staleness
CSR Client-Side Rendering	In browser (JS)	User dashboards post-login, real-time data	Poor SEO, slow initial load

Code Examples (App Router):

```
// SSR — runs on every request
async function Page() {
  const data = await fetch(url, { cache: 'no-store' });
  ...
}

// SSG — runs once at build time
async function Page() {
  const data = await fetch(url, { cache: 'force-cache' });
  ...
}

// ISR — rebuild every 60 seconds
export const revalidate = 60;

// CSR — runs in browser
'use client';
function Page() {
  const [data, setData] = useState(null);
  useEffect(() => { fetchData().then(setData); }, []);
}
```

■ **Tip:** You can mix strategies in one Next.js app — even on the same page using different components!

Q3 What are Server Components in Next.js? Advantages & Limitations?

Server Components (RSC) are React components that render **exclusively on the server**. They were introduced in React 18 and are the **default** in Next.js App Router.

Advantages:

- ■ **Zero client JS** — no JavaScript sent to browser for these components
- ■ **Direct data access** — read databases, filesystems, call APIs without exposing secrets
- ■ **Smaller bundle** — heavy dependencies (markdown parsers, SDKs) stay on server
- ■ **Better performance** — data fetching happens close to the source
- ■ **Secure** — API keys, DB connections never reach the client
- ■ **Async by default** — can use await directly in the component body

Limitations:

- ■ Cannot use **useState, useEffect**, or other React hooks
- ■ Cannot access **browser APIs** (window, localStorage, etc.)
- ■ Cannot add **event listeners** (onClick, onChange, etc.)
- ■ Cannot use **Context** (as providers)

```
// Server Component (default in App Router)
```

```
async function UserProfile({ userId }) {
  // Direct DB access — safe!
  const user = await db.users.findById(userId);
  return <div>{user.name}</div>;
}
```

■ **Tip:** Think of Server Components as the 'template' layer — they fetch data and pass it to Client Components for interactivity.

Q4 What are Client Components? When should you use them?

Client Components are marked with the **"use client"** directive. They render on the **server first** (for initial HTML), then **hydrate** on the client to become interactive.

Use Client Components when you need:

- **Interactivity** — onClick, onChange, form submissions
- **React Hooks** — useState, useEffect, useRef, useContext
- **Browser APIs** — localStorage, window, navigator, geolocation
- **Real-time updates** — WebSockets, subscriptions
- **Animations** — framer-motion, CSS transitions triggered by JS

```
'use client'; // Must be at the TOP of the file

import { useState } from 'react';

export function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}
```

■ **Tip:** Best practice: Push Client Components to the LEAVES of your component tree. Keep parents as Server Components to minimize the client bundle.

■■ **Warning:** 'use client' marks a BOUNDARY — everything imported inside a Client Component also becomes client-side, even if not marked.

Q5 What is the difference between App Router and Pages Router?

Next.js has two routing systems. The **App Router** (introduced in v13) is the modern approach, while **Pages Router** is the legacy system.

Feature	App Router	Pages Router
Directory	app/	pages/
Default Component Type	Server Components	Client Components
Layouts	Nested layouts (layout.js)	Manual _app.js
Data Fetching	async/await in components	getServerSideProps / getStaticProps
Loading States	loading.js convention	Manual implementation
Error Handling	error.js convention	Manual try/catch
Streaming	Built-in with Suspense	Not supported
Server Actions	Supported	Not supported
Stability	Stable (v13.4+)	Stable, but legacy
Learning Curve	Steeper	Easier for beginners

■ **Tip:** New projects should use App Router. Both can coexist in the same project during migration.

Q6 What is Hydration? What causes hydration mismatch errors?

Hydration is the process where React takes the static HTML rendered on the server and attaches JavaScript event listeners to make it interactive on the client.

The Hydration Process:

1. **Server renders** — Next.js renders components to HTML string on the server
2. **HTML sent** — Browser receives and displays the static HTML immediately
3. **JS loads** — React JavaScript bundle downloads in the background
4. **Hydration** — React 'attaches' to existing DOM, adding interactivity

Common Causes of Hydration Mismatch:

- ■ **Date/Time rendering (new Date())**
  - ■ Fix: Use suppressHydrationWarning or render on client only
- ■ **Math.random() or crypto values**
  - ■ Fix: Generate on server and pass as props
- ■ **Browser-only checks (typeof window)**
  - ■ Fix: Use useEffect for client-only code
- ■ **User-specific data in SSR**
  - ■ Fix: Fetch after hydration with useEffect
- ■ **Conditional rendering based on cookies/auth state**

■ Fix: Use middleware or defer to client

```
// ■ Causes hydration error
function Component() {
  return <div>{new Date().toString()}</div>;
}

// ■ Fixed with useEffect
'use client';
function Component() {
  const [date, setDate] = useState('');
  useEffect(() => setDate(new Date().toString()), []);
  return <div>{date}</div>;
}
```

Q7 What is Streaming in Next.js and how does it improve performance?

Streaming allows Next.js to send HTML to the browser in chunks as it's generated, rather than waiting for the entire page to be ready. It's built on HTTP streaming and React's Suspense.

How it works:

Without streaming, a page with slow data fetching makes the user wait for ALL data before seeing ANYTHING. With streaming, the shell (layout, nav) renders immediately, and slow sections stream in as they complete.

```
// app/dashboard/page.tsx
import { Suspense } from 'react';

export default function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>  {/* Renders immediately */}
      <Suspense fallback={<Skeleton />}>
        <SlowDataComponent />  {/* Streams in when ready */}
      </Suspense>
      <Suspense fallback={<Spinner />}>
        <AnotherSlowComponent />  {/* Streams in independently */}
      </Suspense>
    </div>
  );
}
```

The loading.js Convention:

Creating a loading.js file in any route segment automatically wraps the page in Suspense and shows the loading UI while data fetches.

```
// app/dashboard/loading.tsx
export default function Loading() {
  return <DashboardSkeleton />;
}
```

■ **Tip:** Streaming dramatically improves perceived performance. Users see content faster even if total load time is similar.

Q8 How does data fetching work in the App Router?

In the App Router, data fetching is done directly inside **async Server Components** using the extended fetch API or ORM/database calls.

```
// SSG — cached forever (build time)
const data = await fetch(url, { cache: 'force-cache' });

// SSR — fresh on every request
const data = await fetch(url, { cache: 'no-store' });

// ISR — revalidate every 60 seconds
const data = await fetch(url, { next: { revalidate: 60 } });

// ISR — revalidate by tag
const data = await fetch(url, { next: { tags: ['products'] } });
// Then revalidate from server action:
revalidateTag('products');
```

Parallel vs Sequential Fetching:

```
// ■ Sequential — slow (waterfall)
const user = await getUser(id);      // waits...
const posts = await getPosts(id);    // then waits...

// ■ Parallel — fast
const [user, posts] = await Promise.all([
  getUser(id),
  getPosts(id),
]);
```

■ **Tip:** Next.js automatically deduplicates identical fetch() calls within the same request — safe to call the same API in multiple components.

Q9 What is Route Segment Configuration?

Route Segment Configuration allows you to control rendering and caching behavior per route using exported constants from layout.ts or page.ts files.

```
// page.tsx — Route Segment Config

// Control rendering mode
export const dynamic = 'auto';        // default
export const dynamic = 'force-dynamic'; // always SSR
export const dynamic = 'force-static';  // always SSG
export const dynamic = 'error';       // fail if dynamic

// ISR revalidation interval (seconds)
export const revalidate = 3600; // 1 hour
export const revalidate = false; // cache forever
export const revalidate = 0;    // no cache (SSR)

// Fetch strategy for all fetches in segment
export const fetchCache = 'force-cache';
```

```
// Runtime environment
export const runtime = 'edge';   // Edge Runtime
export const runtime = 'nodejs'; // Node.js Runtime (default)

// Streaming timeout
export const maxDuration = 30;  // seconds
```

Q10 What is Middleware in Next.js? Real-world use cases?

Middleware runs **before a request is completed**, at the edge, allowing you to modify request/response or redirect users. It runs on every request matching its config.

```
// middleware.ts (root of project)
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token');

  // Redirect unauthenticated users
  if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
};
```

Real-World Use Cases:

- **Authentication Guard** — Redirect unauthenticated users to login page
- **Authorization / RBAC** — Check user role and redirect if insufficient permissions
- **A/B Testing** — Route users to different page variants based on cookie
- **Internationalization** — Detect user locale and redirect to /en/, /fr/, etc.
- **Rate Limiting** — Block excessive requests from an IP
- **Feature Flags** — Enable/disable features per user or region
- **Bot Detection** — Block or challenge suspicious traffic patterns
- **Geo-routing** — Redirect users to region-specific pages

■■ **Important:** Middleware runs on the Edge Runtime — it must not use Node.js APIs. Keep it fast and lightweight.

Q11 How does the next/image component optimize images?

next/image is a drop-in replacement for the HTML img tag that provides automatic image optimization without any configuration.

- **Format conversion**: Automatically serves WebP/AVIF to supported browsers (30-80% smaller)

- **Lazy loading**: Images load only when they enter the viewport (default behavior)
- **Responsive sizes**: Generates srcset for different screen sizes automatically
- **Blur placeholder**: Shows a blurred version while the full image loads
- **Size optimization**: Resizes images server-side to the exact display size
- **CDN caching**: Optimized images are cached at CDN level
- **CLS prevention**: Requires width/height to prevent Cumulative Layout Shift

```
import Image from 'next/image';

// Local image (auto-detects dimensions)
<Image src={heroImg} alt='Hero' priority />

// Remote image (requires dimensions)
<Image
  src='https://example.com/photo.jpg'
  alt='Photo'
  width={800}
  height={600}
  placeholder='blur'
  blurDataURL='...'
/>

// Responsive fill
<div style={{ position: 'relative', height: '400px' }}>
  <Image src={img} alt='...' fill style={{ objectFit: 'cover' }} />
</div>
```

Q12 How would you implement authentication in a Next.js application?

Authentication in Next.js typically involves 3 layers: **session management**, **route protection**, and **API security**.

Recommended approach with NextAuth.js / Auth.js:

```
// 1. Setup — auth.ts
import NextAuth from 'next-auth';
import GitHub from 'next-auth/providers/github';

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [GitHub],
  callbacks: {
    authorized({ auth, request: { nextUrl } }) {
      const isLoggedIn = !!auth?.user;
      const isOnDashboard = nextUrl.pathname.startsWith('/dashboard');
      if (isOnDashboard) return isLoggedIn;
      return true;
    },
  },
});

// 2. Middleware protection
export { auth as middleware } from '@/auth';

// 3. Server Component — get session
const session = await auth();
if (!session) redirect('/login');

// 4. Client Component
const { data: session } = useSession();
```

Authorization (checking permissions):

```
// In Server Component
const session = await auth();
if (session?.user?.role !== 'admin') {
  return <AccessDenied />;
}
```

Q13 What are Server Actions? How are they different from API Routes?

Server Actions are async functions that run on the server but can be called directly from Client Components — like calling a function, but it executes server-side. They're defined with **'use server'**.

```
// actions/user.ts
'use server';

import { db } from '@/lib/db';
import { revalidatePath } from 'next/cache';

export async function createUser(formData: FormData) {
  const name = formData.get('name') as string;
  await db.users.create({ data: { name } });
  revalidatePath('/users'); // Refresh the page data
}

// Usage in Client Component
'use client';
import { createUser } from '@/actions/user';

function CreateUserForm() {
  return (
    <form action={createUser}>
      <input name='name' />
      <button type='submit'>Create</button>
    </form>
  );
}
```

Feature	Server Actions	API Routes
Definition	'use server' directive	app/api/route.ts file
Calling	Directly as function / form action	fetch('/api/endpoint')
Type Safety	Full TypeScript end-to-end	Manual typing of fetch calls
Boilerplate	Minimal	More setup required
External access	Not directly accessible	Public HTTP endpoint
Best for	Form submissions, mutations	Third-party webhooks, public APIs

Q14 Difference between cookies(), sessionStorage, and localStorage?

Feature	cookies()	sessionStorage	localStorage
Location	Server + Client	Client (browser) only	Client (browser) only
Persistence	Configurable (session or expiry date)	Until tab is closed	Until manually cleared
Capacity	~4KB per cookie	~5MB	~5-10MB
Access in Next.js	cookies() in Server Components, Middleware	Client JS only	Client JS only
Sent to server	Yes — with every HTTP request	No	No
Security	Can be HttpOnly (no JS access)	Accessible via JS	Accessible via JS
Best for	Auth tokens, sessions, CSRF tokens	Temporary UI state per tab	Persistent user preferences

```
// Next.js cookies() — Server Component
import { cookies } from 'next/headers';

async function ServerComp() {
  const cookieStore = await cookies();
  const token = cookieStore.get('auth-token');
}

// Set cookie in Server Action
'use server';
import { cookies } from 'next/headers';
async function login() {
  const cookieStore = await cookies();
  cookieStore.set('auth-token', token, {
    httpOnly: true,
    secure: true,
    sameSite: 'lax',
  });
}
```

Q15 How does caching work in Next.js? force-cache, no-store, revalidate.

Next.js has **4 caching layers** that work together to maximize performance:

Request Memoization: Same fetch() calls in one render tree deduplicated automatically

Data Cache: fetch() results persisted across requests and deployments (CDN-like)

Full Route Cache: Entire rendered HTML + RSC payload cached on server

Router Cache: Client-side cache of visited routes for instant navigation

Cache Control via fetch():

```
// force-cache: Cache forever (SSG behavior)
await fetch(url, { cache: 'force-cache' });

// no-store: Never cache (SSR behavior)
await fetch(url, { cache: 'no-store' });

// Time-based revalidation (ISR)
await fetch(url, { next: { revalidate: 3600 } });

// Tag-based revalidation
await fetch(url, { next: { tags: ['user', 'products'] } });

// Manually purge cache (in Server Action)
import { revalidateTag, revalidatePath } from 'next/cache';
revalidateTag('products'); // purge all fetches tagged 'products'
revalidatePath('/products'); // purge the /products route cache
```

Q16 What techniques improve performance in a Next.js application?

- **Server Components by default** — Eliminate client JS for non-interactive parts — biggest win
- **Streaming + Suspense** — Show content progressively, avoid waterfall blocking
- **next/image** — Automatic WebP, lazy loading, responsive srcset
- **next/font** — Self-host fonts, eliminate external font requests, no CLS
- **Dynamic imports** — Code-split heavy components: dynamic(() => import('./Heavy'))
- **Route-level code splitting** — Automatic per page in Next.js
- **next/link prefetching** — Prefetches pages on link hover (automatic)
- **Parallel data fetching** — Promise.all() to avoid sequential waterfalls
- **ISR over SSR** — Cache pages and revalidate periodically instead of SSR
- **Edge Runtime for Middleware** — Run auth/redirects geographically close to users
- **Bundle analysis** — @next/bundle-analyzer to find large imports
- **React.memo / useMemo** — Prevent unnecessary re-renders in Client Components

```
// Dynamic import — load only when needed
import dynamic from 'next/dynamic';

const HeavyChart = dynamic(() => import('./Chart'), {
  loading: () => <ChartSkeleton />,
  ssr: false, // Don't SSR this component
});

// next/font — zero layout shift
import { Inter } from 'next/font/google';
const inter = Inter({ subsets: ['latin'] });
```

Q17 What are common causes of hydration errors and how do you fix them?

Hydration errors occur when the **server-rendered HTML doesn't match** what React tries to render on the client during hydration.

Cause	Problem	Fix
typeof window checks	Renders undefined on server, 'object' on client	Move to useEffect
new Date() / Date.now()	Different timestamps server vs client	Use useEffect or suppressHydrationWarning
Math.random()	Different values each render	Generate server-side, pass as prop
Browser extensions	Extensions inject DOM nodes	suppressHydrationWarning on root
Invalid HTML nesting	inside	Fix the HTML structure

Locale formatting	Intl.NumberFormat differs	Force locale explicitly
OS/browser detection	UA string unavailable server-side	Detect post-hydration

```
// ■ Fix 1: useEffect for client-only rendering
const [mounted, setMounted] = useState(false);
useEffect(() => setMounted(true), []);
if (!mounted) return null;

// ■ Fix 2: suppressHydrationWarning (small mismatches only)
<time suppressHydrationWarning>
  {new Date().toLocaleDateString()}
</time>

// ■ Fix 3: dynamic import with ssr:false
const ClientOnlyComp = dynamic(() => import('./Comp'), { ssr: false });
```

Q18 What are Dynamic Routes and how are they implemented?

Dynamic Routes allow pages to be generated from dynamic data using **bracket notation** in the filename.

```
// File structure → URL pattern
app/blog/[slug]/page.tsx        → /blog/anything
app/shop/[...categories]/page.tsx  → /shop/a/b/c
app/docs/[[...slug]]/page.tsx   → /docs OR /docs/a/b
app/[lang]/[country]/page.tsx   → /en/us  /fr/ca
```

```
// app/blog/[slug]/page.tsx
interface Props {
  params: Promise<{ slug: string }>;
}

export async function generateStaticParams() {
  const posts = await getPosts();
  return posts.map(post => ({ slug: post.slug }));
}

export default async function BlogPost({ params }: Props) {
  const { slug } = await params;
  const post = await getPost(slug);
  return <article>{post.content}</article>;
}
```

generateStaticParams() is the App Router equivalent of getStaticPaths(). It pre-generates routes at build time for SSG.

■ **Tip:** Without generateStaticParams(), dynamic routes are rendered on-demand (SSR). Adding it enables SSG for those routes.

Q19 How would you implement Role-Based Access Control (RBAC) in Next.js?

RBAC restricts access based on user roles (admin, editor, user). Implement it at **3 layers** for complete security:

```
// Layer 1: Middleware — fast edge-level check
import { auth } from '@/auth';

export default auth((req) => {
  const role = req.auth?.user?.role;
  const isAdminRoute = req.nextUrl.pathname.startsWith('/admin');
```

```
  if (isAdminRoute && role !== 'admin') {
    return Response.redirect(new URL('/unauthorized', req.url));
  }
});

// Layer 2: Server Component — UI-level check
export default async function AdminPage() {
  const session = await auth();
  if (session?.user?.role !== 'admin') notFound();
  return <AdminDashboard />;
}

// Layer 3: Server Action — data mutation check
'use server';
export async function deleteUser(userId: string) {
  const session = await auth();
  if (session?.user?.role !== 'admin') {
    throw new Error('Unauthorized');
  }
  await db.users.delete({ where: { id: userId } });
}
```

■■ **Security:** Always validate roles in Server Actions and API Routes — never trust client-side role checks alone. Defense in depth!

Q20 A Next.js page loads slowly. How do you investigate and optimize it?

Use a **systematic debugging approach** — measure first, optimize second.

1. **Measure**: Use Chrome DevTools (Network, Performance), Lighthouse, and Core Web Vitals
2. **Check Rendering Mode**: Is this SSR? SSG? CSR? Wrong choice = avoidable overhead
3. **Analyze Bundle**: Run ANALYZE=true next build to find large client-side packages
4. **Profile Data Fetching**: Add timing logs to identify slow DB queries or APIs
5. **Check Waterfalls**: Are fetches sequential when they could be parallel?
6. **Examine Images**: Are large unoptimized images being served?
7. **Check Client Components**: Are large components needlessly marked 'use client'?
8. **Network tab**: Look for 3rd party scripts, large JSON responses, unoptimized assets

Common Fixes Checklist:

- ☐ Switch SSR → SSG/ISR if data doesn't change per-user
- ☐ Add Suspense boundaries to stream slow sections
- ☐ Use dynamic() to lazy-load heavy Client Components
- ☐ Replace useEffect data fetching with Server Component fetching
- ☐ Add database indexes for slow queries
- ☐ Use Promise.all() for parallel fetches
- ☐ Optimize images with next/image
- ☐ Move third-party scripts to next/script with strategy='lazyOnload'
- ☐ Enable HTTP/2 and CDN for static assets

```
// Diagnose slow data fetching
async function getData() {
  console.time('db-query');
  const data = await db.heavyQuery();
  console.timeEnd('db-query');
  return data;
}

// next.config.js — enable bundle analyzer
// ANALYZE=true next build
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});
module.exports = withBundleAnalyzer({});
```

Quick Reference Cheatsheet

`'use client'`	Mark a Client Component boundary	`'use server'`	Mark a Server Action or Server file
`loading.tsx`	Automatic Suspense wrapper for a route	`error.tsx`	Error boundary for a route segment
`layout.tsx`	Shared UI that persists across child routes	`not-found.tsx`	Rendered when notFound() is called
`route.ts`	API Route handler (GET, POST, etc.)	`middleware.ts`	Runs before every matching request
`generateStaticParams()`	Pre-generate dynamic route params at build	`revalidatePath(path)`	Purge cached route from server cache
`revalidateTag(tag)`	Purge all fetches with a specific tag	`notFound()`	Trigger 404 response from Server Component
`redirect(url)`	Redirect from Server Component or Action	`cookies()`	Read/write cookies in Server Components
`headers()`	Read request headers in Server Components		